LAPORAN PRAKTIKUM

STRUKTUR DATA

“SINGLE LINKED LIST”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya, laporan praktikum dengan judul “SINGLE LINKED LIST” dapat diselesaikan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran dalam memahami konsep dasar pemrograman khususnya Single Linked List pada Bahasa Java.

Pada kesempatan ini, penulis menyampaikan terimakasih kepada:

1. Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan bimbingan dan arahan.
2. Kak Zahran Azaria Anvaya selaku asisten praktikum kelas A yang telah membantu pelaksanaan praktikum.
3. Teman-teman mahasiswa yang telah membantu dan memberi dukungan serta berdiskusi bersama dalam menyelesaikan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, krirtik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai pemrograman serta struktur data pada Java.

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II **3**
2.1 Langkah Kerja Praktikum 3
2.2 Analisis Hasil dan Pembahasan 14
BAB III **17**
3.1 Kesimpulan 17
DAFTAR PUSTAKA .....

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data adalah kunci utama dalam mengelola informasi secara efisien di dalam program. Salah satu struktur yang paling fleksibel untuk digunakan adalah **Single Linked List**. Berbeda dengan array yang ukurannya kaku (statis), Single Linked List memungkinkan kita mengelola data secara dinamis karena setiap elemen (node) saling terhubung melalui alamat memori.

Keunggulan utama dari struktur ini terletak pada kemudahannya saat kita perlu menambah atau menghapus data tanpa harus mengatur ulang seluruh indeks, seperti yang terjadi pada array. Karena fleksibilitasnya inilah, pemahaman tentang Linked List menjadi pondasi penting sebelum mempelajari struktur data yang lebih rumit.

Dalam praktikum kali ini, saya mengimplementasikan operasi-operasi dasar Single Linked List menggunakan Java, meliputi proses penambahan, penghapusan, hingga pencarian data. Tujuannya adalah agar teori mengenai manipulasi pointer dan node ini dapat dipahami secara praktis melalui kode program.

1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data Single Linked List.
2. Memahami cara kerja node dan hubungan antar node pada Single Linked List.
3. Mengimplementasikan operasi penambahan data pada Single Linked List.
4. Mengimplementasikan operasi penghapusan data pada Single Linked List.
5. Memahami proses traversal dan pencarian data pada Single Linked List menggunakan bahasa pemrograman Java.
6. Melatih kemampuan logika dan pemahaman algoritma dalam pengolahan struktur data linear.

1.3 Manfaat Praktikum

1. Mahasiswa dapat memahami konsep dan cara kerja Single Linked List secara lebih mendalam.
2. Mahasiswa dapat meningkatkan kemampuan pemrograman menggunakan bahasa Java dalam implementasi struktur data.

3. Mahasiswa dapat memahami proses manipulasi data, seperti penambahan, penghapusan, dan pencarian data pada linked list.
4. Mahasiswa dapat melatih kemampuan berpikir logis dan sistematis dalam menyelesaikan permasalahan pemrograman.
5. Praktikum ini dapat menjadi dasar untuk mempelajari struktur data lain yang lebih kompleks pada materi selanjutnya.

BAB II
PEMBAHASAN

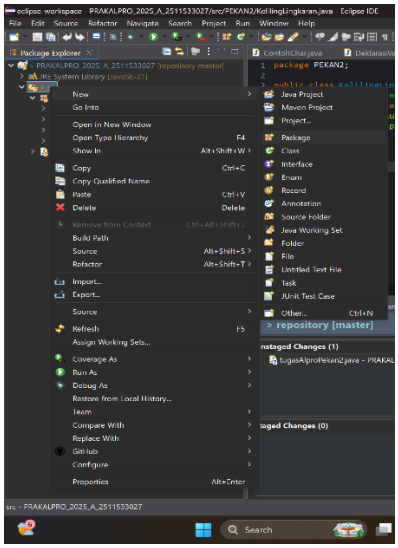
2.1 Langkah Kerja Praktikum

1. Persiapan Awal

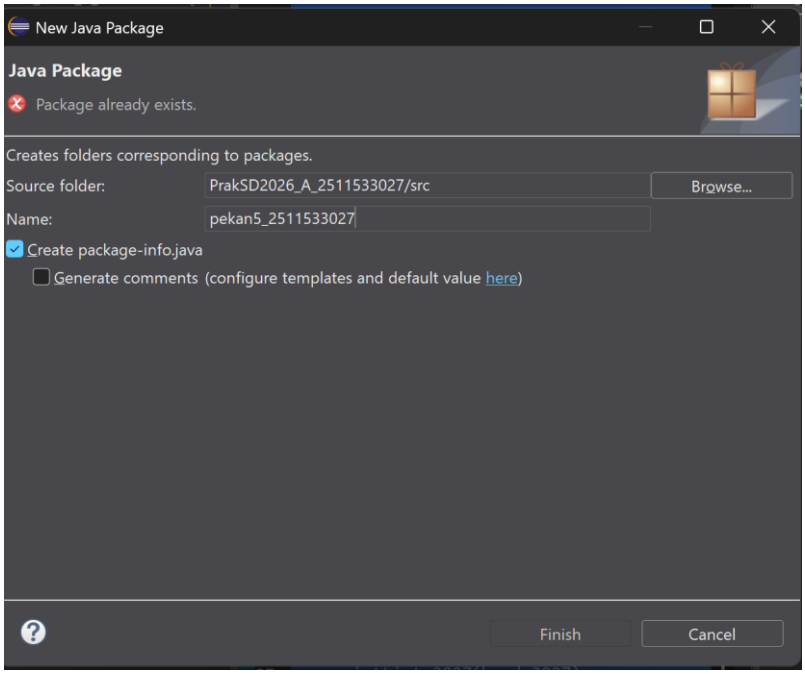
   a) Buka apl Eclipse IDE di laptop atau di komputer.

2. Membuat Package

   a) Klik kanan pada folder **src**, lalu pilih **New**, terakhir klik **Package**.

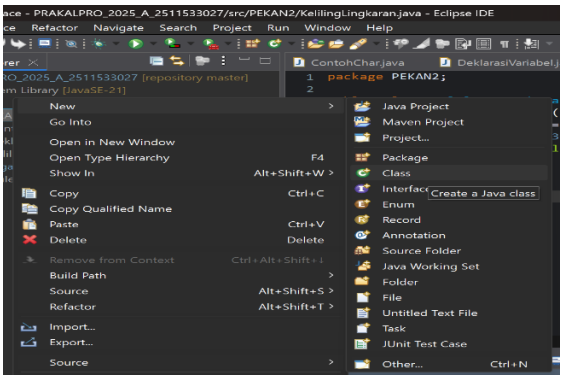


   b) Beri nama package **pekan5_NIM**.



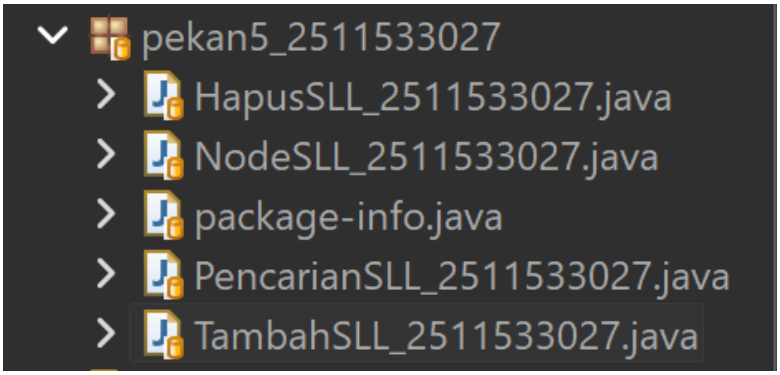
3. Membuat Class Baru untuk Setiap Percobaan

   a) Klik kanan pada package **pekan4_NIM**, lalu pilih **New**, terakhir klik **Class**.



   b) Buat beberapa kelas sesuai percobaan.

   - HapusSLL_2511533027.java
   - NodeSLL_2511533027.java
   - Pencarian_2511533027.java
   - TambahSLL_2511533027.java



4. Menulis Kode Program
   1. HapusSLL_2511533027.java
      - Program ini digunakan untuk memahami proses penghapusan data pada struktur data *Single Linked List*.
      - Program memiliki beberapa method untuk menghapus node, yaitu menghapus node di awal (*head*), node terakhir, dan node pada posisi tertentu.
      - Method deleteHead_3027() digunakan untuk menghapus node pertama pada linked list dengan memindahkan head ke node berikutnya.
      - Method removeLastNode_3027() digunakan untuk menghapus node terakhir dengan menelusuri linked list hingga node sebelum terakhir.
      - Method deleteNode_3027() digunakan untuk menghapus node berdasarkan posisi yang ditentukan.
      - Program juga memiliki method printList_3027() untuk menampilkan seluruh isi linked list.
      - Pada method main, program membuat linked list secara manual kemudian melakukan beberapa operasi penghapusan data.
      - Hasil akhir program berupa tampilan linked list sebelum dan sesudah proses penghapusan dilakukan.

```
package pekan5_2511533027;

public class HapusSLL_2511533027 {
   // fungsi untuk menghapus head
   public  static  NodeSLL_2511533027  deleteHead_3027(NodeSLL_2511533027
head_3027) {
         // jika SLL kosong
         if (head_3027 == null)
               return null;
         // pindahkan head ke node berikutnya
         head_3027 = head_3027.next_3027;
         // return head baru
         return head_3027;     }
   // fungsi mengahpus node terakhir SLL
   public                    static                    NodeSLL_2511533027
removeLastNode_3027(NodeSLL_2511533027 head_3027) {
```

```
        // jika list kosong, return null
        if (head_3027 == null) {
            return null;
        }
        // jika list satu node, hapus node dan return null
        if (head_3027.next_3027 == null) {
            return null;
        }
        // temukan node terakhir ke dua
        NodeSLL_2511533027 secondLast = head_3027;
        while (secondLast.next_3027.next_3027 != null) {
            secondLast = secondLast.next_3027;
        }
        // hapus node terakhir
        secondLast.next_3027 = null;
        return head_3027;      }

// fungsi menghapus node di posisi tertentu
    public  static  NodeSLL_2511533027  deleteNode_3027(NodeSLL_2511533027
head_3027, int position_3027) {
        NodeSLL_2511533027 temp_3027 = head_3027;
        NodeSLL_2511533027 prev_3027 = null;
        // jika linked list null
        if (temp_3027 == null)
            return head_3027;
        // kasus 1: head dihapus
        if (position_3027 == 1) {
            head_3027 = temp_3027.next_3027;
            return head_3027;
        }
        // kasus 2: menghapus node di tengah
        // telusuri ke node yang dihapus
    for (int i = 1; temp_3027 != null && i < position_3027; i++) {
        prev_3027 = temp_3027;
        temp_3027 = temp_3027.next_3027;    }
        // jika ditemukan, hapus node
        if (temp_3027 != null) {
            prev_3027.next_3027 = temp_3027.next_3027;
        } else {
            System.out.println("Data tidak ada");     }
```

```
        return head_3027;      }

   // fungsi mencetak SLL
        public     static     void     printList_3027(NodeSLL_2511533027
head_3027) {
                NodeSLL_2511533027 curr_3027 = head_3027;
                while (curr_3027.next_3027 != null) {
                        System.out.print(curr_3027.data_3027+"-->");
                        curr_3027 = curr_3027.next_3027; }
                if (curr_3027.next_3027==null) {
                        System.out.print(curr_3027.data_3027);  }
                System.out.println();
        }

                // kelas main
                public static void main(String[] args) {
                        // buat  SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
                        NodeSLL_2511533027      head_3027      =      new
NodeSLL_2511533027(1);
                        head_3027.next_3027 = new NodeSLL_2511533027(2);
                        head_3027.next_3027.next_3027        =        new
NodeSLL_2511533027(3);
                        head_3027.next_3027.next_3027.next_3027   =   new
NodeSLL_2511533027(4);
                        head_3027.next_3027.next_3027.next_3027.next_3027
= new NodeSLL_2511533027(5);

   head_3027.next_3027.next_3027.next_3027.next_3027.next_3027   =   new
NodeSLL_2511533027(6);
                        // cetak list awal
                        System.out.println("List awal: ");
                        printList_3027(head_3027);
                        // hapus head
                        head_3027 = deleteHead_3027(head_3027);
                        System.out.println("List  setelah  head  dihapus:
");
                        printList_3027(head_3027);
                        // hapus node terakhir
                        head_3027 = removeLastNode_3027(head_3027);
```

```
                        System.out.println("List setelah simpul terakhir
dihapus: ");
                        printList_3027(head_3027);
                        // deleting node at position 2
                        int position_3027 = 2;
                        head_3027 = deleteNode_3027(head_3027,
position_3027);
                        // print list after deletion
                        System.out.println("List setelah posisi 2
dihapus: ");
                        printList_3027(head_3027);
                }
}
```

Output:

```
<terminated> HapusSLL_2511533027 [Java Application]
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

2. NodeSLL_2511533027.java
   - Program ini digunakan untuk membentuk node pada struktur data Single Linked List.
   - Kelas NodeSLL_2511533027 memiliki atribut data_3027 untuk menyimpan data dan next_3027 sebagai pointer menuju node berikutnya.
   - Konstruktor pada kelas digunakan untuk menginisialisasi data pada node saat objek dibuat.
   - Nilai next_3027 diatur null sebagai tanda bahwa node belum terhubung dengan node lain.
   - Kelas ini menjadi dasar utama dalam pembuatan dan pengelolaan Single Linked List.

```
package pekan5_2511533027;
```

```java
public class NodeSLL_2511533027 {
   // node bagian data
   int data_3027;
   // Pointer ke node berikutnya
   NodeSLL_2511533027 next_3027;
   // Konstruktor menginisialisasi node dengan data
   public NodeSLL_2511533027(int data_3027) {
         this.data_3027 = data_3027;
         this.next_3027 = null;
   }

}
```

3. PencarianSLL_2511533027.java
   - Program ini digunakan untuk memahami proses traversal dan pencarian data pada struktur data *Single Linked List*.
   - Method traversal_3027() digunakan untuk menampilkan seluruh isi linked list secara berurutan dari node pertama hingga terakhir.
   - Method searchKey_3027() digunakan untuk mencari data tertentu dalam linked list.
   - Program melakukan penelusuran node satu per satu menggunakan pointer hingga data ditemukan atau mencapai akhir linked list.
   - Jika data yang dicari ditemukan, program akan menampilkan keterangan "Ketemu".
   - Jika data tidak ditemukan, program akan menampilkan keterangan "Tidak ada".
   - Pada method main, program membuat linked list secara manual kemudian melakukan proses traversal dan pencarian data.
   - Hasil akhir program berupa tampilan isi linked list dan hasil pencarian data tertentu.

```java
package pekan5_2511533027;

public class PencarianSLL_2511533027 {
   static boolean searchKey_3027(NodeSLL_2511533027 head_3027, int
key_3027) {
         NodeSLL_2511533027 curr_3027 = head_3027;
```

```
        while (curr_3027 != null) {
                if (curr_3027.data_3027 == key_3027)
                        return true;
                curr_3027 = curr_3027.next_3027;  }
        return false;  }
    public static void traversal_3027(NodeSLL_2511533027 head_3027) {
        // mulai dari head
        NodeSLL_2511533027 curr_3027 = head_3027;
        // telusuri sampai pointer null
        while (curr_3027 != null) {
                System.out.print(" " + curr_3027.data_3027);
                curr_3027 = curr_3027.next_3027;  }
        System.out.println();            }
        public static void main(String[] args) {
                NodeSLL_2511533027 head = new NodeSLL_2511533027(14);
                head.next_3027 = new NodeSLL_2511533027(21);
                head.next_3027.next_3027 = new NodeSLL_2511533027(13);
                head.next_3027.next_3027.next_3027         =         new
NodeSLL_2511533027(30);
                head.next_3027.next_3027.next_3027.next_3027    =    new
NodeSLL_2511533027(10);
                System.out.print("Penulusuran SLL: ");
                traversal_3027(head);
                // data yang akan dicari
                int key_3027 = 30;
                System.out.print("Cari data " +key_3027+ " = ");
                if (searchKey_3027(head, key_3027))
                        System.out.println("Ketemu");
                else
                        System.out.println("Tidak ada");
        }
    }
```

Output:

```
<terminated> PencarianSLL_2511533027 [Java Applicati
Penulusuran SLL:  14 21 13 30 10
Cari data 30 = Ketemu
```

4. TambahSLL_2511533027.java

- Program ini digunakan untuk memahami proses penambahan node pada struktur data *Single Linked List*.
- Program memiliki beberapa method untuk menambahkan node di depan, di belakang, dan pada posisi tertentu.
- Method insertAtFront_3027() digunakan untuk menambahkan node di awal linked list.
- Method insertAtEnd_3027() digunakan untuk menambahkan node di akhir linked list dengan menelusuri node terakhir terlebih dahulu.
- Method insertPos_3027() digunakan untuk menambahkan node pada posisi tertentu di dalam linked list.
- Program juga memiliki method printList_3027() untuk menampilkan isi linked list setelah proses penambahan data.
- Pada method main, program membuat linked list awal kemudian melakukan beberapa proses penambahan node.
- Hasil akhir program berupa tampilan linked list sebelum dan sesudah penambahan data dilakukan.

```
package pekan5_2511533027;

public class TambahSLL_2511533027 {
  public static NodeSLL_2511533027 insertAtFront_3027(NodeSLL_2511533027 head_3027, int value_3027) {
    NodeSLL_2511533027 new_node = new NodeSLL_2511533027(value_3027);
    new_node.next_3027 = head_3027;
    return new_node;
  }

  // fungsi menambahkan node diakhir SLL
  public static NodeSLL_2511533027 insertAtEnd_3027(NodeSLL_2511533027 head_3027, int value_3027) {
    NodeSLL_2511533027 newNode_3027 = new NodeSLL_2511533027(value_3027);

    if (head_3027 == null) {
      return newNode_3027;
    }
```

```
    NodeSLL_2511533027 last = head_3027;

    while (last.next_3027 != null) {
      last = last.next_3027;
    }

    last.next_3027 = newNode_3027;
    return head_3027;
  }

  static NodeSLL_2511533027 GetNode_3027(int data_3027) {
    return new NodeSLL_2511533027(data_3027);
  }

  static      NodeSLL_2511533027      insertPos_3027(NodeSLL_2511533027
headNode_3027, int position_3027, int value_3027) {
    NodeSLL_2511533027 head_3027 = headNode_3027;

    if (position_3027 < 1)
      System.out.print("invalid position");

    if (position_3027 == 1) {
      NodeSLL_2511533027 new_node = new NodeSLL_2511533027(value_3027);
      new_node.next_3027 = head_3027;
      return new_node;
    } else {
      while (position_3027-- != 0) {
        if (position_3027 == 1) {
          NodeSLL_2511533027 newNode_3027 = GetNode_3027(value_3027);
          newNode_3027.next_3027 = headNode_3027.next_3027;
          headNode_3027.next_3027 = newNode_3027;
          break;
        }
        headNode_3027 = headNode_3027.next_3027;
      }
      if (position_3027 != 1)
        System.out.print("Posisi di luar jangkauan");
    }
    return head_3027;
```

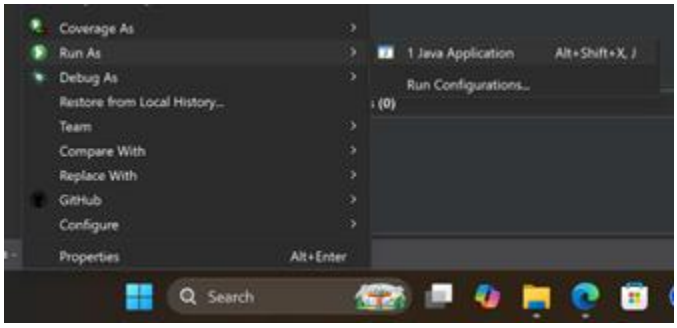
```
  }

  public static void printList_3027(NodeSLL_2511533027 head_3027) {
    NodeSLL_2511533027 curr = head_3027;
    while (curr.next_3027 != null) {
      System.out.print(curr.data_3027 + "-->");
      curr = curr.next_3027;
    }
    if (curr.next_3027 == null) {
      System.out.print(curr.data_3027);
      System.out.println();
    }
  }

  public static void main(String[] args) {
    NodeSLL_2511533027 head_3027 = new NodeSLL_2511533027(2);
    head_3027.next_3027 = new NodeSLL_2511533027(3);
    head_3027.next_3027.next_3027 = new NodeSLL_2511533027(5);
    head_3027.next_3027.next_3027.next_3027 = new
NodeSLL_2511533027(6);

    System.out.print("Senarai berantai awal:");
    printList_3027(head_3027);

    System.out.print("tambah 1 simpul di depan: ");
    int data_3027 = 1;
    head_3027 = insertAtFront_3027(head_3027, data_3027);
    printList_3027(head_3027);

    System.out.print("tambah 1 simpul di belakang: ");
    int data2 = 7;
    head_3027 = insertAtEnd_3027(head_3027, data2);
    printList_3027(head_3027);

    System.out.print("tambah 1 simpul ke data 4: ");
    int data3 = 4;
    int pos = 4;
    head_3027 = insertPos_3027(head_3027, pos, data3);
    printList_3027(head_3027);
  }
```

```
}
```

Output:

```
<terminated> TambahSLL_2511533027 [Java Application] C:\Users\hp\.p2\pool\p
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

5. Kompilasi dan Menjalankan Program



2.2 Analisis Hasil dan Pembahasan

1. HapusSLL_2511533027.java
   a) Hasil: Program berhasil melakukan proses penghapusan node pada Single Linked List. Penghapusan dapat dilakukan pada bagian awal linked list (*head*), bagian akhir, maupun pada posisi tertentu sesuai input yang diberikan. Setelah proses penghapusan dilakukan, isi linked list berubah sesuai node yang telah dihapus dan hasilnya dapat ditampilkan dengan benar menggunakan method printList_3027().
   b) Pembahasan: Pada program ini diterapkan beberapa operasi dasar penghapusan data pada Single Linked List. Method deleteHead_3027() bekerja dengan memindahkan pointer head ke node berikutnya sehingga node pertama tidak lagi terhubung dengan linked list. Method removeLastNode_3027() digunakan untuk menghapus node terakhir dengan cara menelusuri node hingga mencapai node sebelum terakhir. Selain itu, method deleteNode_3027() digunakan untuk menghapus node berdasarkan posisi tertentu dengan bantuan pointer sementara dan pointer sebelumnya. Dari hasil percobaan dapat dilihat bahwa setiap proses penghapusan memengaruhi hubungan antar node pada linked list.
2. NodeSLL_2511533027.java

a) Hasil: Program berhasil membentuk node yang digunakan dalam struktur data Single Linked List. Setiap node dapat menyimpan data serta memiliki pointer yang menghubungkan node tersebut dengan node berikutnya. Node-node yang dibuat kemudian dapat digunakan pada program lain untuk proses penambahan, penghapusan, maupun pencarian data.
b) Pembahasan: Kelas NodeSLL_2511533027 merupakan dasar utama dalam pembentukan Single Linked List. Di dalam kelas ini terdapat atribut data_3027 untuk menyimpan nilai data dan next_3027 sebagai penghubung antar node. Konstruktor digunakan agar data dapat langsung diinisialisasi saat objek dibuat. Nilai awal pointer diatur null karena node belum terhubung ke node lain. Dengan adanya kelas node ini, proses manipulasi data pada linked list dapat dilakukan dengan lebih mudah.

3. PencarianSLL_2511533027.java
   a) Hasil: Program berhasil menampilkan seluruh isi Single Linked List dan melakukan pencarian data sesuai nilai yang dimasukkan. Pada percobaan yang dilakukan, data yang dicari dapat ditemukan dan program menampilkan keterangan “Ketemu”. Proses traversal juga berjalan dengan baik sehingga seluruh data pada linked list dapat ditampilkan secara berurutan.
   b) Pembahasan: Program ini menggunakan metode traversal untuk menelusuri linked list dari node pertama hingga terakhir. Method traversal_3027() berfungsi untuk menampilkan semua data yang terdapat pada linked list secara berurutan. Sedangkan method searchKey_3027() digunakan untuk mencari data tertentu dengan membandingkan nilai pada setiap node. Proses pencarian dilakukan satu per satu hingga data ditemukan atau pointer mencapai null. Cara kerja ini menunjukkan bahwa pencarian pada Single Linked List dilakukan secara linear karena data harus ditelusuri secara berurutan.
4. TambahSLL_2511533027.java
   a) Hasil: Program berhasil melakukan penambahan node pada Single Linked List di beberapa posisi, yaitu di awal, di akhir, dan di posisi tertentu. Setelah proses penambahan dilakukan, linked list menampilkan data baru sesuai urutan yang diharapkan. Seluruh perubahan data dapat ditampilkan dengan baik menggunakan method printList_3027().
   b) Pembahasan: Pada program ini dilakukan implementasi operasi penambahan node pada Single Linked List. Method insertAtFront_3027() digunakan untuk

menambahkan node di bagian depan dengan menjadikan node baru sebagai head. Method insertAtEnd_3027() bekerja dengan menelusuri linked list hingga node terakhir, kemudian menghubungkan node baru di bagian akhir. Selain itu, method insertPos_3027() digunakan untuk menyisipkan node pada posisi tertentu dengan mengatur hubungan pointer antar node. Dari hasil percobaan terlihat bahwa proses penambahan node memerlukan pengaturan pointer yang tepat agar susunan linked list tetap terhubung dengan baik.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat dipahami bahwa Single Linked List merupakan struktur data linear yang terdiri dari beberapa node yang saling terhubung melalui pointer. Pada praktikum ini dilakukan beberapa operasi dasar, seperti menambahkan node, menghapus node, menampilkan isi linked list, dan mencari data tertentu menggunakan bahasa pemrograman Java.

Dari hasil percobaan, setiap operasi pada Single Linked List memiliki cara kerja yang berbeda tergantung posisi node yang diproses. Proses penambahan dan penghapusan data dilakukan dengan mengatur hubungan antar node melalui pointer `next`. Selain itu, proses traversal dan pencarian data dilakukan dengan menelusuri node satu per satu mulai dari head hingga node terakhir.

Melalui praktikum ini, pemahaman mengenai penggunaan node, pointer, serta alur kerja Single Linked List menjadi lebih jelas. Praktikum ini juga membantu melatih ketelitian dalam mengatur hubungan antar node agar linked list tetap berjalan dengan baik setelah dilakukan perubahan data.

DAFTAR PUSTAKA

[1] Wahyudi, *Materi Struktur Data: Single Linked List*. Padang: PPT Perkuliahan, 2026.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA, USA: Pearson, 2012.

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

“SINGLE LINKED LIST”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas rahmat dan karunia-Nya sehingga laporan tugas praktikum Struktur Data ini dapat diselesaikan dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW.

Laporan tugas ini disusun untuk memenuhi tugas praktikum Struktur Data dengan topik Single Linked List menggunakan studi kasus Simulasi Antrian Rumah Sakit. Pada praktikum ini, penulis membuat program Java yang dapat mengelola data Website seperti menambah, menampilkan, dan menghapus data.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan.

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I **1**
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
1.4 Instruksi Tugas 2
BAB II **4**
2.1 Kelas ADT (Pasien_NIMLengkap) 4
2.2 Kelas Driver (RumahSakit_NIMLengkap) 5
BAB III **11**
3.1 Kesimpulan 11
DAFTAR PUSTAKA .....

BAB I
PENDAHULUAN

1.1 Latar Belakang

Sistem antrian merupakan bagian yang tidak terpisahkan dari pelayanan publik, seperti halnya pada pendaftaran pasien di rumah sakit. Agar pelayanan berjalan adil, rumah sakit harus menerapkan prinsip kedatangan di mana pasien yang datang lebih awal akan dilayani terlebih dahulu. Dalam pemrograman, logika ini dapat diwujudkan secara efektif menggunakan struktur data **Single Linked List**. Berbeda dengan *array* yang memiliki batasan kapasitas sejak awal, Single Linked List memungkinkan sistem antrian bersifat dinamis; kita bisa menambah atau menghapus data pasien sesuai kondisi riil di lapangan tanpa khawatir kekurangan memori.

Dalam implementasinya, setiap pasien direpresentasikan sebagai sebuah simpul (node) yang saling terhubung. Penggunaan Single Linked List sangat relevan untuk simulasi ini karena mendukung prinsip **FIFO (First In, First Out)** secara efisien melalui manipulasi pointer. Dengan mengarahkan pointer pada setiap node, kita dapat melakukan operasi penambahan pasien di akhir antrian maupun pemanggilan pasien di urutan paling depan dengan cepat. Melalui tugas praktikum ini, konsep abstrak Single Linked List diterapkan ke dalam program Java untuk memecahkan masalah antrian rumah sakit, mulai dari proses pendaftaran, pencarian data pasien, hingga pengelolaan urutan pelayanan secara sistematis.

1.2 Tujuan

Tujuan:

1. Memahami konsep dasar struktur data Single Linked List.
2. Memahami penerapan konsep FIFO pada sistem antrian rumah sakit.
3. Mengimplementasikan operasi dasar Single Linked List menggunakan bahasa pemrograman Java.
4. Membuat program simulasi antrian rumah sakit dengan fitur tambah, hapus, tampil, pencarian, dan pengecekan data pasien.
5. Melatih kemampuan logika dan pemecahan masalah dalam pembuatan program berbasis struktur data.

1.3 Manfaat

Manfaat:

1. Mahasiswa dapat memahami cara kerja Single Linked List secara lebih nyata melalui simulasi antrian rumah sakit.
2. Mahasiswa dapat memahami penggunaan pointer dan hubungan antar node dalam struktur data linear.
3. Mahasiswa dapat meningkatkan kemampuan pemrograman Java, terutama dalam implementasi struktur data.
4. Mahasiswa dapat melatih kemampuan berpikir logis dan sistematis dalam menyelesaikan permasalahan pemrograman.
5. Tugas ini dapat menjadi dasar untuk mempelajari struktur data dan algoritma yang lebih kompleks pada materi berikutnya.

1.4 Instruksi Tugas

1. Aturan Penamaan
    1. Nama Kelas : Gunakan NIM lengkap.
       Contoh : Pasien_2411531005 dan RumahSakit_2411531005.
    2. Nama Variabel dan Method : Setiap variabel dan method wajib menggunakan 4 digit terakhir NIM.
       Contoh : String namaPasien_1005, String penyakit_1005.
    3. Pointer next : Variabel pointer ke node berikutnya juga wajib diberi akhiran 4 digit NIM.
       Contoh : Pasien_2411531005 next_1005.
2. Struktur Program
    1. Kelas ADT Pasien (Pasien_NIMLengkap)

       Atribut :

       - Nama Pasien
       - Keluhan / Penyakit
       - Nomor Antrian (int)
       - Pointer next ke node Pasien berikutnya

       Method :

       - Constructor untuk menginisialisasi semua atribut.
       - Selektor (Getter) untuk setiap atribut.
       - Mutator (Setter) untuk setiap atribut termasuk pointer next.

2. Kelas Driver (RumahSakit_NIMLengkap)
   - Daftarkan Pasien (Insert at Tail) : Menambahkan node pasien baru di akhir linked list. Jika list kosong, node baru langsung menjadi head. Gunakan sebuah variabel counter untuk menyimpan nomor antrian terakhir. Setiap kali fungsi Daftarkan Pasien dipanggil, nomor antrian akan bertambah secara otomatis (auto-increment).
   - Panggil Pasien (Delete Head) : Menghapus node terdepan dan menampilkan data pasien yang dipanggil. Head digeser ke node berikutnya.
   - Tampilkan Antrian (Display) : Menelusuri linked list dari head hingga null dan menampilkan semua data pasien beserta posisinya.
   - Cari Pasien (Search) : Pencarian dilakukan berdasarkan nama pasien secara Case Insensitive (tidak membedakan huruf besar dan kecil). Tampilkan pesan jika tidak ditemukan.
   - Cek Status Antrian : Menampilkan jumlah total pasien dan informasi pasien terdepan. Tampilkan pesan jika list kosong.
3. Contoh Output

   === Antrian Rumah Sakit NIM: 2411531005 ===

   1. Daftarkan Pasien (Insert)
   2. Panggil Pasien (Delete Head)
   3. Tampilkan Antrian (Display)
   4. Cari Pasien (Search)
   5. Cek Status Antrian
   6. Keluar

   Pilihan: 1

   Masukkan Nama Pasien : Budi Santoso

   Masukkan Keluhan : Demam

   Pasien berhasil didaftarkan! Nomor Antrian: 1

BAB II
PEMBAHASAN

2.1 Kelas ADT (Pasien_NIMLengkap)

Kelas Pasien_2511533027 digunakan untuk merepresentasikan data pasien pada sistem antrian rumah sakit. Pada kelas ini terdapat beberapa atribut, yaitu nama pasien, keluhan, nomor antrian, dan pointer next_3027 yang digunakan untuk menghubungkan node pasien dengan node berikutnya pada Single Linked List. Kelas ini juga dilengkapi dengan constructor untuk menginisialisasi data pasien saat objek dibuat. Selain itu, tersedia method getter dan setter yang berfungsi untuk mengambil serta mengubah nilai atribut pasien. Kelas ini menjadi dasar utama dalam pembentukan struktur data Single Linked List pada program simulasi antrian rumah sakit.

```java
package pekan5_2511533027;

public class Pasien_2511533027 {
    String namaPasien_3027;
    String penyakit_3027;
    int nomorAntrian_3027;

    // pointer ke pasien berikutnya
    Pasien_2511533027 next_3027;

    // constructor
    public Pasien_2511533027 (String namaPasien_3027, String penyakit_3027, int nomorAntrian_3027) {
        this.namaPasien_3027 = namaPasien_3027;
        this.penyakit_3027 = penyakit_3027;
        this.nomorAntrian_3027 = nomorAntrian_3027;
    }

    // getter
    public String getNamaPasien_3027() {
    }

    public String getPenyakit_3027() {
        return penyakit_3027;
    }

    public int getNomorAntrian_3027() {
        return nomorAntrian_3027;
    }

    // setter
    public void setNamaPasien_3027(String namaPasien_3027) {
        this.namaPasien_3027 = namaPasien_3027;
    }

    public void setPenyakit_3027(String penyakit_3027) {
        this.penyakit_3027 = penyakit_3027;
    }

    public void setNomorAntrian_3027(int nomorAntrian_3027) {
        this.nomorAntrian_3027 = nomorAntrian_3027;
    }

}
```

2.2 Kelas Driver (RumahSakit_NIMLengkap)

Kelas RumahSakit_2511533027 merupakan kelas utama yang digunakan untuk menjalankan program simulasi antrian rumah sakit berbasis Single Linked List. Pada kelas ini terdapat beberapa method yang digunakan untuk mengelola data antrian pasien, seperti menambahkan pasien ke akhir antrian (*insert at tail*), memanggil pasien terdepan (*delete head*), menampilkan seluruh data antrian, mencari pasien berdasarkan nama, serta mengecek status antrian. Program juga menerapkan konsep FIFO (*First In First Out*), sehingga pasien yang lebih dahulu mendaftar akan dipanggil terlebih dahulu. Pada method main, program menampilkan menu interaktif yang memungkinkan pengguna memilih operasi yang ingin dijalankan sesuai kebutuhan.

```java
package pekan5_2511533027;

import java.util.Scanner;

public class RumahSakit_2511533027 {
    static Pasien_2511533027 head_3027 = null;
    static int counter_3027 = 0;

    // method insert at tail
    public static void daftarkanPasien_3027(String nama_3027, String penyakit_3027) {
        counter_3027++;
        Pasien_2511533027 pasienBaru_3027 = new Pasien_2511533027(nama_3027, penyakit_3027, counter_3027);

        // jika antrian (linked list) kosong
        if (head_3027 == null) {
            head_3027 = pasienBaru_3027;
        } else {
            Pasien_2511533027 temp_3027 = head_3027;
            while (temp_3027.next_3027 != null) {
                temp_3027 = temp_3027.next_3027;
            }
            // sambungkan node terakhir ke node baru
            temp_3027.next_3027 = pasienBaru_3027;
        }

        System.out.println("Pasien Berhasil Didaftarkan!");
        System.out.println("Nomor Antrian: " + pasienBaru_3027.nomorAntrian_3027);
    }
    // method delete head
    public static void panggilPasien_3027() {
        // jika antrian kosong
        if (head_3027 == null) {
            System.out.println("Antrian Masih Kosong.");
            return;
        }
        System.out.println("Pasien Dipanggil: ");
        System.out.println("Nomor Antrian   : " + head_3027.nomorAntrian_3027);
        System.out.println("Nama Pasien     : " + head_3027.namaPasien_3027);
        System.out.println("Keluhan         : " + head_3027.penyakit_3027);
        // geser head ke node berikutnya
        head_3027 = head_3027.next_3027;
    }
```

```java
    // method display
    public static void tampilkanAntrian_3027() {
        if (head_3027 == null) {
            System.out.println("Antrian Masih Kosong.");
            return;
        }

        Pasien_2511533027 temp_3027 = head_3027;
        int posisi_3027 = 1;
        System.out.println("=== DAFTAR ANTRIAN PASIEN ===");
        while (temp_3027 != null) {
            System.out.println("Posisi Antrian: " + posisi_3027);
            System.out.println("Nomor Antrian : " + temp_3027.nomorAntrian_3027);
            System.out.println("Nama Pasien   : "+ temp_3027.namaPasien_3027);
            System.out.println("Keluhan       :" + temp_3027.penyakit_3027);
            System.out.println();

            temp_3027 = temp_3027.next_3027;
            posisi_3027++;
        }
    }

    // method search
    public static void cariPasien_3027(String namaCari_3027) {
        if (head_3027 == null) {
            System.out.println("Antrian Masih kosong.");
            return;
        }

        Pasien_2511533027 temp_3027 = head_3027;
        boolean ketemu_3027 = false;
        while (temp_3027 != null) {
            if (temp_3027.namaPasien_3027.equalsIgnoreCase(namaCari_3027)) {
                System.out.println("Pasien Ditemukan!");
                System.out.println("Nomor Antrian: " + temp_3027.nomorAntrian_3027);
                System.out.println("Nama Pasien  : " + temp_3027.namaPasien_3027);
                System.out.println("Keluhan      : " + temp_3027.penyakit_3027);
                ketemu_3027 = true;
                break;
            }
            temp_3027 = temp_3027.next_3027;
        }
```

```java
        if (!ketemu_3027) {
            System.out.println("Pasien Tidak Ditemukan.");
        }
    }

    // method cek status antrian
    public static void cekStatusAntrian_3027() {
        if (head_3027 == null) {
            System.out.println("Antrian Masih Kosong.");
            return;
        }

        Pasien_2511533027 temp_3027 = head_3027;
        int jumlah_3027 = 0;
        while(temp_3027 != null) {
            jumlah_3027++;
            temp_3027 = temp_3027.next_3027;
        }
        System.out.println("Jumlah Pasien Dalam Antrian: ");
        System.out.println("Nomor Antrian              : " + head_3027.nomorAntrian_3027);
        System.out.println("Nama Pasien                : " + head_3027.namaPasien_3027);
        System.out.println("Keluhan                    : " + head_3027.penyakit_3027);
    }

    // main program
    public static void main (String[] args) {
        Scanner input_3027 = new Scanner(System.in);
        int pilihan_3027;
        do {
            System.out.println();
            System.out.println("=== Antrian Rumah Sakit NIM: 2511533027 ===");
            System.out.println("1. Daftarkan Pasien");
            System.out.println("2. Panggil Pasien");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Cari Pasien");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");

            pilihan_3027 = input_3027.nextInt();
            input_3027.nextLine();
```

```java
            pilihan_3027 = input_3027.nextInt();
            input_3027.nextLine();

            switch (pilihan_3027) {
            case 1:
                System.out.println("Masukkan Nama Pasien: ");
                String nama_3027 = input_3027.nextLine();
                System.out.println("Masukkan Keluhan: ");
                String penyakit_3027 = input_3027.nextLine();
                daftarkanPasien_3027(nama_3027, penyakit_3027);
                break;
            case 2:
                panggilPasien_3027();
                break;
            case 3:
                tampilkanAntrian_3027();
                break;
            case 4:
                System.out.print("Masukkan Nama Pasien Yang Dicari: ");
                String cari_3027 = input_3027.nextLine();
                cariPasien_3027(cari_3027);
                break;
            case 5:
                cekStatusAntrian_3027();
                break;
            case 6:
                System.out.println("Program Selesai.");
                break;
            default:
                System.out.println("Pilihan Tidak Tersedia.");
            }
        }
        while (pilihan_3027 != 6);
        input_3027.close();
    }

}
```

Output:

```
=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien:
rara
Masukkan Keluhan:
mual
Pasien Berhasil Didaftarkan!
Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien:
salma
Masukkan Keluhan:
mual
Pasien Berhasil Didaftarkan!
Nomor Antrian: 2
```

```
=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== DAFTAR ANTRIAN PASIEN ===
Posisi Antrian: 1
Nomor Antrian : 1
Nama Pasien   : rara
Keluhan       :mual

Posisi Antrian: 2
Nomor Antrian : 2
Nama Pasien   : salma
Keluhan       :mual


=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah Pasien Dalam Antrian:
Nomor Antrian              : 1
Nama Pasien                : rara
Keluhan                    : mual
```

```
=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien Yang Dicari: rara
Pasien Ditemukan!
Nomor Antrian: 1
Nama Pasien  : rara
Keluhan      : mual

=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Pasien Dipanggil:
Nomor Antrian   : 1
Nama Pasien     : rara
Keluhan         : mual
```

```
=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== DAFTAR ANTRIAN PASIEN ===
Posisi Antrian: 1
Nomor Antrian : 2
Nama Pasien   : salma
Keluhan       :mual


=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah Pasien Dalam Antrian:
Nomor Antrian              : 2
Nama Pasien                : salma
Keluhan                    : mual

=== Antrian Rumah Sakit NIM: 2511533027 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan:
```

BAB III
PENUTUP

3.1 Kesimpulan

Dari tugas yang telah dibuat, dapat dipahami bahwa Single Linked List merupakan struktur data yang bekerja dengan cara menghubungkan setiap node menggunakan pointer. Struktur data ini cocok digunakan pada simulasi antrian rumah sakit karena data pasien bisa ditambah dan dihapus secara dinamis sesuai kebutuhan. Sistem antrian pada program juga sudah menerapkan konsep FIFO (First In First Out), sehingga pasien yang datang lebih dulu akan dipanggil lebih dulu.

Pada program ini berhasil dibuat beberapa fitur utama, seperti menambahkan pasien ke antrian, memanggil pasien, menampilkan daftar antrian, mencari data pasien, dan melihat status antrian. Semua proses tersebut dilakukan dengan memanfaatkan hubungan antar node pada Single Linked List.

Melalui tugas ini, pemahaman tentang cara kerja linked list menjadi lebih mudah dipahami, terutama pada bagian penggunaan pointer, traversal, dan manipulasi data. Selain itu, tugas ini juga membantu melatih logika pemrograman serta ketelitian dalam mengatur hubungan antar node agar program dapat berjalan dengan baik.

DAFTAR PUSTAKA

[1] Wahyudi, *Materi Struktur Data: Single Linked List*. Padang: PPT Perkuliahan, 2026.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA, USA: Pearson, 2012.